

Integration Test API Fails

I ran the integration test suite and a large number of failures appear to come from API implementation inconsistencies rather than from the tests themselves.

Main Categories of Failures

1. Incorrect HTTP Status Codes Returned

Several endpoints return different status codes than expected.

Expected `201 Created` but API returns `200 OK`

Affected tests:

- `OfferRequest_ValidOwner_ReturnsSuccess`
- `ApproveRequest_ValidOwner_ReturnsSuccess`
- `CancelRequest_ValidRenter_ReturnsNoContent`
- `DenyRequest_UnauthorizedUser_ReturnsForbidden`
- `OfferRequest_NotOwner_ReturnsForbidden`
- `ApproveRequest_UnauthorizedUser_ReturnsForbidden`
- `DenyRequest_ValidOwner_ReturnsNoContent`

This suggests the API creates resources successfully but returns `200` instead of `201`.

Expected `409 Conflict` but API returns `400 BadRequest`

Affected tests:

- `Register_WithDuplicateEmail_ReturnsConflict`
- `Register_WithDuplicateUsername_ReturnsConflict`

Likely validation logic returns `BadRequest` before duplicate checks, or duplicate registration is intentionally implemented as `400`.

Expected `401 Unauthorized` but API returns `400 BadRequest`

Affected tests:

- `ChangePassword_WithWrongCurrentPassword_ReturnsUnauthorized`
- `Login_WithValidCredentials_ReturnsOk` (returns `401`)

Possible authentication / validation mismatch.

Expected `404 NotFound` but API returns `400 BadRequest`

Affected tests:

- `ChangePassword_WithNonExistentId_ReturnsNotFound`
- `Update_WithMissingNotification_ReturnsNotFound`

Expected `500 InternalServerError` but API returns `404 NotFound`

Affected tests:

- `Delete_WhenGameDoesNotExist_ReturnsInternalServerError`
- `Update_WhenGameDoesNotExist_ReturnsInternalServerError`

This may actually indicate the API behavior is correct and the tests should be updated to expect `404`.

1. Authentication / Authorization Configuration Errors

Affected tests:

- `Update_WhenUserIsNotOwner_ReturnsForbidden`
- `Delete_WhenUserIsOwner_ReturnsOk`
- `Delete_WhenUserIsNotOwner_ReturnsForbidden`
- `Delete_WhenGameHasActiveRentals_ReturnsBadRequest`
- `Update_WhenDataIsInvalid_ReturnsBadRequest`
- `Update_WhenUserIsOwner_ReturnsNoContent`

Error: `No authentication handler is registered for the scheme 'You are not authorized to ...'`

This looks like `Unauthorized("message")` or `Forbid("message")` is being used incorrectly.

Example of problematic usage:

```
return Forbid("You are not authorized to update this game.");
```

ASP.NET interprets the string as an authentication scheme name.

Should probably be replaced with something like:

```
return StatusCode(403, "You are not authorized to update this game.");
```

or:

```
return Forbid();
```

1. EF Core `ExecuteDelete()` InMemory Provider Failure

Affected tests:

- `CancelRequest_ValidRenter_ReturnsNoContent`
- `DenyRequest_ValidOwner_ReturnsNoContent`

Problem: `ExecuteDelete()` cannot be translated by the EF Core InMemory provider.

Current repository code likely contains:

```
.ExecuteDelete()
```

Recommended fix:

```
var notifications = await context.Notifications
    .Where(...)
    .ToListAsync();

context.Notifications.RemoveRange(notifications);

await context.SaveChangesAsync();
```

1. Empty Response Body / Invalid JSON

Affected tests:

- CreateConversation_ReturnsCreated
- SendMessage_ReturnsPersistedMessage
- CreateConversation_ThenGetById_ReturnsConversation
- GetPaymentHistory_ReturnsSortedByDateDescending
- GetPaymentHistory_ReturnsOnlyUserPayments

Error: The input does not contain any JSON tokens

This usually means:

- endpoint returns empty body
- endpoint returns plain text instead of JSON
- serialization failed
- response stream is empty

The tests are trying to deserialize JSON but the API response body is empty.

1. Data Persistence / Update Logic Issues

Affected test:

- UpdatePayment_ThenGet_ReturnsUpdatedPayment

Expected:

```
PaidAmount = 999
```

Actual:

```
PaidAmount = 10
```

Looks like update logic is not persisting changes.

1. User Filtering / Test Isolation Problems

Affected tests:

- `GetUsersExcept_ReturnsAllOtherUsers`
- `GetUsersExcept_WithNonExistentUserId_ReturnsAllUsers`

Expected counts:

- 2
- 3

Actual counts:

- 5
- 6

Possible causes:

- database not cleaned between tests
- shared seeded users
- endpoint filtering issue

Summary

Most failures appear to be API-side issues related to:

- inconsistent status codes
- incorrect auth handling
- EF Core InMemory incompatibilities
- missing JSON responses
- persistence/update logic
- test database isolation

A smaller subset may require test expectation updates where the API behavior is actually more appropriate (e.g. returning `404` instead of `500`).